

Project Plan

CS 4530 – Mobile App Development

Keaton Voordeckers – u1410421

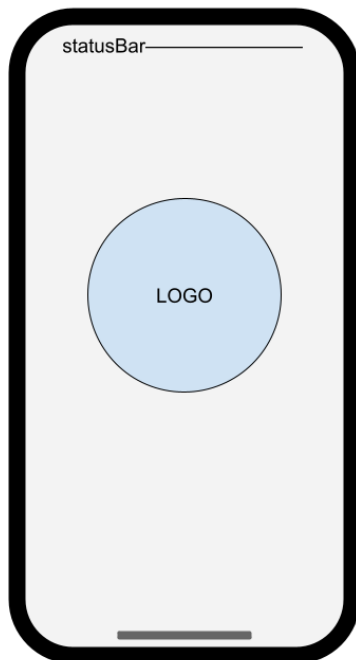
Jack Wakefield - u1185200

Graham Taggart - u0578188

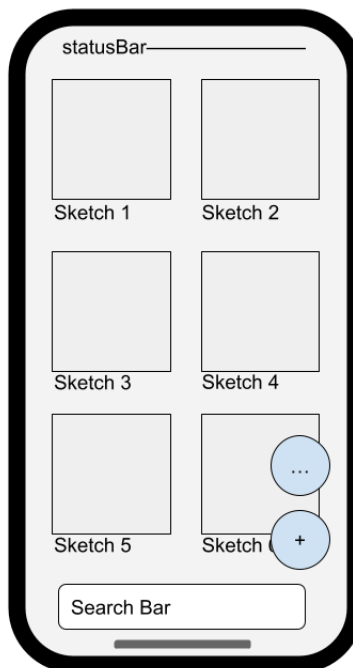
[Requirements](#)

Wireframes:

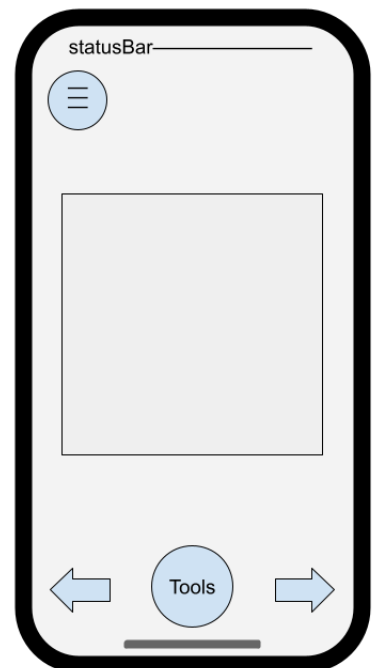
Splash screen:



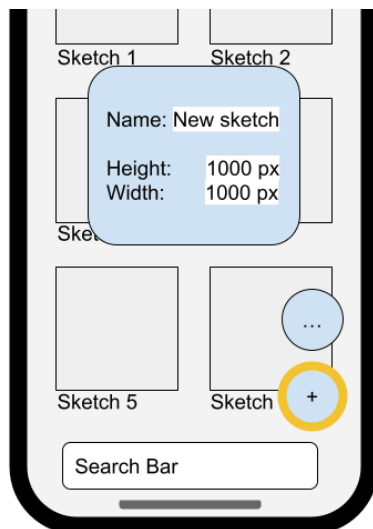
Main screen:



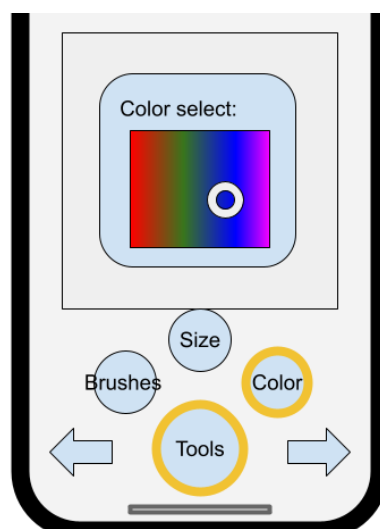
Drawing Screen:



New sketch button pressed:



Tools then Color buttons pressed:



Task Breakdown:

1) Splash Screen (Jack Wakefield)

- Create a splash screen layout with a logo in the middle or a loading icon.
- Potentially use Jetpack Compose Animations to create a startup animation.
- Create a logo/icon to display on the splash screen.
- Test that splash-screen allows time for app to load the main screen and that the app doesn't crash on startup.

2) Main screen - drawings created by the user (Jack Wakefield)

- Create a main screen layout with a LazyColumn and Rows to display images/scroll, a button for multiple selection and sharing/deleting, a button and toast for creating a new sketch, and a search bar.
- Create a select function, share function, delete function, create sketch function, and search function.
- Test for scrolling, test for selection properly opens image screen layout, test for multiple delete functionality, test for properly sized components.

3) Create new drawings/edit existing ones (Keaton Voordeckers & Jack Wakefield)

- Implement or use the predefined 'Canvas' UI element to draw on via listeners to click and track the x & y position on the canvas when click occurs.
- Create backing list to represent stored Canvases. Canvases can be added, removed, and edited (Resize, Crop, Rotate, Draw, Erase, Undo, Redo).
- Create backing stack of Canvas's to store changes and allow for redo/undo
- Implement form of JSON saving to save Canvases to device and load new Canvases to be drawn on.
- Test for resizing (when a resize is called does the Canvas size change to reflect the input width and length).
- Test for Undo/Redo (when undo or redo function is called, does the stack size appropriately reflect the change. EX: Undo->Pop->StackSize--)

4) Manage cloud backups or sharing functionality (Graham Taggart)

- Create a backup screen layout with a list of the user's drawings, backup and restore buttons for each drawing, and a progress indicator for uploads/downloads.
- Create an uploadDrawing function (converts drawing to PNG and saves to Firebase Storage/S3), listBackups function, downloadDrawing function, and deleteBackup function.
- Test that uploads appear in cloud storage, test that restore recovers drawings after reinstalling the app, test that upload failures show an error, test that users cannot access other users' backups.

5) Share images with other apps (Graham Taggart)

- Create a share button on the canvas screen and gallery items that opens the Android system share sheet.
- Create a renderCanvasToBitmap function (draws canvas paths to a PNG), saveImageToCache function, and shareImage function.
- Test that the exported image has the correct size and content, test that the saved PNG opens correctly, test that sharing to Gmail/Messages delivers the image without crashing.

6) AI-powered image analysis (Keaton Voordeckers)

- Create a UI element dedicated to displaying the text output of the image analysis.
- Create a button to enable and begin image analysis on the current Canvas.
- Allow for importing images from camera roll into Canvas for image analysis.
- Test output for a variety of imported images and drawings on Canvas via human verification. (Does output reasonably recognize the input drawing/image and does it describe it reasonably accurate)

7) Automated Testing (Keaton Voordeckers, Jack Wakefield & Graham Taggart)

Drawing logic tests:

- Test creating an empty drawing.
- Test adding a stroke to a drawing.
- Test selecting different pen or brush colors.
- Test selecting different brush sizes or shapes.

- Test erasing a stroke.
- Test clearing the canvas.
- Test undo and redo, if implemented.
- Test that drawing coordinates are handled correctly.
- Test invalid or unusual input, such as an empty stroke.

Saving and loading tests:

- Test saving a new drawing.
- Test loading an existing drawing.
- Test updating an existing drawing.
- Test deleting a drawing.
- Test that saved strokes, colors, and brush settings remain intact across undo/redo.
- Test that the main screen displays all saved drawings.

Screen and navigation tests:

- Test that the splash screen appears when the app launches.
- Test that the splash screen navigates to the main screen.
- Test that creating a new drawing opens the drawing editor.
- Test that selecting an existing drawing opens it for editing.
- Test that the backup/sharing screen can be opened.
- Test back-button navigation.
- Test important screens on phone and tablet configurations.
- Test screen rotation or other configuration changes.

Sharing and backup tests:

- Test that sharing creates the correct Android share intent.
- Test that the shared item has an appropriate image type, such as image/png.
- Test that a drawing is successfully converted into an image.
- Test backup behavior with a fake or test data source.

AI image-analysis tests:

- Test importing a supported image.
- Test rejecting or reporting unsupported files.
- Test passing the image to the analysis component.
- Test converting AI results into the text displayed by the app.
- Test displaying text representing detected object labels and confidence values.
- Test empty results, errors, and timeouts.

- Keep a small set of sample images for controlled testing.